ARTIFICIAL INTELLIGENCE

SCENARIO-BASED ASSIGNMENT

Question 1 – Greedy Best-First Search

a) Strategy and justification

The strategy is Greedy Best-First Search. At each step, the robot chooses the unvisited neighbouring location with the smallest heuristic h(n), which is the straight-line distance to the Auditorium. It ignores the distance already travelled, g(n).

b) Decision trace

Step / current location	Unvisited candidates (straight-line h)	Choice
1 – Gate	Library: 330 m; Canteen: 240 m	Canteen (240 m)
2 – Canteen	Hostel: 160 m; Admin: 110 m	Admin (110 m)
3 – Admin	Auditorium: 0 m	Auditorium (0 m)

c) Route, distance, and optimality

Route: Gate → Canteen → Admin → Auditorium.

Total road distance = 200 + 260 + 120 = 580 m. This is not guaranteed to be the shortest route because the greedy strategy considers only h(n), not the distance already travelled. For example, Gate → Canteen → Hostel → Auditorium is 200 + 100 + 180 = 480 m, which is 100 m shorter.

Python program – Greedy Best-First Search

```
from heapq import heappush, heappop

graph = {
   "Gate": {"Library": 120, "Canteen": 200},
   "Library": {"Gate": 120, "Lab": 150, "Hostel": 300},
   "Canteen": {"Gate": 200, "Hostel": 100, "Admin": 260},
   "Lab": {"Library": 150, "Auditorium": 350},
   "Hostel": {"Library": 300, "Canteen": 100, "Auditorium": 180},
   "Admin": {"Canteen": 260, "Auditorium": 120},
   "Auditorium": {"Lab": 350, "Hostel": 180, "Admin": 120}
}
h = {"Gate": 400, "Library": 330, "Canteen": 240, "Lab": 280,
    "Hostel": 160, "Admin": 110, "Auditorium": 0}

frontier = [(h["Gate"], "Gate", ["Gate"], 0)]
visited = set()
while frontier:
   _, node, path, distance = heappop(frontier)
   if node in visited:
      continue
   visited.add(node)
   if node == "Auditorium":
      print("Route:", " -> ".join(path))
      print("Distance:", distance, "m")
      break
   for nxt, cost in graph[node].items():
      if nxt not in visited:
         heappush(frontier, (h[nxt], nxt, path + [nxt], distance + cost))
```

Question 2 – A* Search

a) Strategy and shortest-path guarantee

The new strategy is A* Search. It evaluates each candidate using f(n) = g(n) + h(n), where g(n) is the road distance from the Gate so far and h(n) is the straight-line estimate to the Auditorium. With an admissible heuristic (never overestimating the true remaining cost), and the usual graph-search handling of repeated states, A* can return an optimal shortest route. The given straight-line values are admissible for this graph.

b) A* trace

Expansion / candidate	g: distance so far (m)	h: straight-line (m)	f = g + h (m)	Decision / note
From Gate → Library	120	330	450	Keep in frontier
From Gate → Canteen	200	240	440	Expand Canteen first (lowest f)
From Canteen → Hostel	300	160	460	Keep in frontier
From Canteen → Admin	460	110	570	Keep in frontier
Frontier: Library	120	330	450	Expand Library next
From Library → Lab	270	280	550	Keep in frontier
From Library → Hostel	420	160	580	Worse than Hostel at g=300; retain better path
Frontier: Hostel	300	160	460	Expand Hostel next
From Hostel → Auditorium	480	0	480	Goal selected; stop

c) Heuristic safety and route comparison

The heuristic is safe because it does not exceed the true shortest remaining road distance. Four examples:

Location	Given h(n) (m)	True shortest remaining distance (m)	Safe?
Canteen	240	280 (Canteen → Hostel → Auditorium)	Yes
Hostel	160	180 (direct to Auditorium)	Yes
Admin	110	120 (direct to Auditorium)	Yes
Lab	280	350 (direct to Auditorium)	Yes

A* route: Gate → Canteen → Hostel → Auditorium. Distance = 200 + 100 + 180 = 480 m. Compared with Question 1's 580 m route, A* saves 100 m. The 480 m route is shortest: the Gate-to-Canteen-to-Hostel-to-Auditorium route costs 480 m, and the other obvious routes through Library/Lab or Admin are no shorter.

Python program – A* Search

```
from heapq import heappush, heappop

graph = {
   "Gate": {"Library": 120, "Canteen": 200},
   "Library": {"Gate": 120, "Lab": 150, "Hostel": 300},
   "Canteen": {"Gate": 200, "Hostel": 100, "Admin": 260},
   "Lab": {"Library": 150, "Auditorium": 350},
   "Hostel": {"Library": 300, "Canteen": 100, "Auditorium": 180},
   "Admin": {"Canteen": 260, "Auditorium": 120},
   "Auditorium": {"Lab": 350, "Hostel": 180, "Admin": 120}
}
h = {"Gate": 400, "Library": 330, "Canteen": 240, "Lab": 280,
    "Hostel": 160, "Admin": 110, "Auditorium": 0}

pq = [(h["Gate"], 0, "Gate")]
parent = {"Gate": None}
```

```
g = {"Gate": 0}
closed = set()

while pq:
    f, cost, node = heappop(pq)
    if node in closed:
        continue
    if node == "Auditorium":
        path = []
        cur = node
        while cur is not None:
            path.append(cur)
            cur = parent[cur]
        path.reverse()
        print("Route:", " -> ".join(path))
        print("Distance:", g[node], "m")
        break
    closed.add(node)
    for nxt, road in graph[node].items():
        new_g = g[node] + road
        if new_g < g.get(nxt, float("inf")):
            g[nxt] = new_g
            parent[nxt] = node
            heappush(pq, (new_g + h[nxt], new_g, nxt))
```

Question 3 – Constraint Satisfaction Problem (CSP)

a) Variables, domains, and constraints

This is a Constraint Satisfaction Problem, specifically a graph-colouring / timetable-scheduling problem.

Variables: W1, W2, W3, W4, W5. Each variable represents the time slot of one workshop.

Initial domains: W1, W2, W5 = {Morning, Afternoon, Evening}; W3 = {Afternoon, Evening}; W4 = {Morning, Afternoon}.

Constraints: conflicting pairs must have different slots: W1 ≠ W2, W1 ≠ W3, W2 ≠ W3, W2 ≠ W4, W3 ≠ W5, W4 ≠ W5. Also W3 ≠ Morning and W4 ≠ Evening.

b) MRV timetable trace

Step	Workshop selected and reason	Assignment	Remaining domains after propagation
1	W3 (2 slots; degree 3, highest among the 2-slot tie)	W3 = Afternoon	W1={Morning, Evening}; W2={Morning, Evening}; W4={Morning, Afternoon}; W5={Morning, Evening}
2	W2 (2 slots; most unscheduled conflicts among tied workshops)	W2 = Morning	W1={Evening}; W4={Afternoon}; W5={Morning, Evening}
3	W4 (1 slot; tie-break by more unscheduled conflicts than W1)	W4 = Afternoon	W1={Evening}; W5={Morning, Evening}
4	W1 (1 remaining slot)	W1 = Evening	W5={Morning, Evening}
5	W5 (last workshop; Morning tried first)	W5 = Morning	All workshops assigned; no domain is empty

Valid timetable

Time slot	Workshops
Morning	W2, W5
Afternoon	W3, W4

Evening	W1

The timetable is valid: none of the listed conflicting pairs share a slot, W3 is not in the Morning, and W4 is not in the Evening.

c) Why MRV helps; failure handling

Minimum Remaining Values (MRV) chooses the most constrained workshop first. This exposes dead ends early and reduces unnecessary search branches. If a workshop has no legal slot left, the current assignment is inconsistent: backtrack to the most recent decision, try its next available slot, and propagate the constraints again.

Python program – CSP backtracking with MRV

```
slots = ["Morning", "Afternoon", "Evening"]
conflicts = {
   "W1": {"W2", "W3"},
   "W2": {"W1", "W3", "W4"},
   "W3": {"W1", "W2", "W5"},
   "W4": {"W2", "W5"},
   "W5": {"W3", "W4"}
}
domains = {
   "W1": slots[:], "W2": slots[:],
   "W3": ["Afternoon", "Evening"],
   "W4": ["Morning", "Afternoon"],
   "W5": slots[:]
}
assignment = {}

def backtrack():
   if len(assignment) == len(domains):
      return True
   unassigned = [w for w in domains if w not in assignment]
   # MRV, then highest degree among unassigned neighbours, then workshop number
   def key(w):
      legal = [s for s in domains[w]
            if all(assignment.get(n) != s for n in conflicts[w])]
      degree = sum(n not in assignment for n in conflicts[w])
      return (len(legal), -degree, int(w[1:]))
   w = min(unassigned, key=key)
   for s in slots:
      if s not in domains[w]:
         continue
      if all(assignment.get(n) != s for n in conflicts[w]):
         assignment[w] = s
         if backtrack():
            return True
         del assignment[w]
   return False

if backtrack():
   for w in sorted(assignment, key=lambda x: int(x[1:])):
      print(w, "=", assignment[w])
else:
   print("No valid timetable exists.")
```

Question 4 – Minimax Search

a) Technique and assumption

The bot should use the Minimax algorithm. It assumes Player O also plays optimally and chooses replies that minimize X's final score, while X chooses moves that maximize the score.

b) Values from leaves to root

Branch	Final scores for X	Value at X node (MAX)	Value at O node (MIN)
X first move 1, O reply 1	4, 7	max(4, 7) = 7	
X first move 1, O reply 2	8, 2	max(8, 2) = 8	min(7, 8) = 7
X first move 2, O reply 1	3, 5	max(3, 5) = 5	
X first move 2, O reply 2	6, 9	max(6, 9) = 9	min(5, 9) = 5
Starting node (X / MAX)	O branch values: 7, 5	max(7, 5) = 7	Root value = 7

c) Best move and principal variation

X should choose its first move 1 because its minimax value is 7, compared with 5 for first move 2. X can guarantee a score of 7. With optimal play, X chooses first move 1, O chooses reply 1 (the branch valued at 7 rather than 8), and X chooses the final score 7 from leaves 4 and 7.

Python program – Minimax

```
# Two X moves; each has two O replies; each reply has two X outcomes.
tree = [
    [[4, 7], [8, 2]],  # X first move 1
    [[3, 5], [6, 9]]   # X first move 2
]
root_values = []
for x_move in tree:
    o_values = [max(final_scores) for final_scores in x_move]
    root_values.append(min(o_values))
    print("O reply values:", o_values, "=> X move value:", min(o_values))
best_move = root_values.index(max(root_values)) + 1
print("Best first move:", best_move)
print("Guaranteed score:", max(root_values))
```

Question 5 – Alpha-Beta Pruning

a) Left-to-right trace

The engine is the MAX player at the root, and the opponent is MIN at each reply node. Alpha is MAX's best guaranteed value so far; beta is MIN's best value in the current branch. A branch can be pruned when beta ≤ alpha.

Move / step	Scores examined	Alpha (root MAX)	Beta (current MIN)	Action
M1	6, 9, 7	6 after M1	M1 ends at 6	Score all three; M1 value = 6
M2	4	6	4	Since beta 4 ≤ alpha 6, skip 8 and 10
M3	7, then 5	6	7 then 5	After 5, beta 5 ≤ alpha 6; skip 12

b) Positions scored and final choice

Six of the nine leaf positions are actually scored: three under M1, one under M2, and two under M3. Three positions are skipped (M2: 8 and 10; M3: 12). The move chosen is M1 with value 6. Full search gives M1=min(6,9,7)=6, M2=min(4,8,10)=4, and M3=min(7,5,12)=5; therefore MAX also chooses M1 with value 6. Alpha-beta pruning preserves the same decision.

c) Move ordering and evaluation function

Move ordering affects pruning efficiency. If a strong MAX move is examined early, alpha becomes high and more later MIN branches can be cut off. Poor ordering may cause fewer cutoffs, although the final minimax result remains unchanged. An evaluation function estimates the desirability of non-terminal positions when the engine cannot search all the way to the end of the game.

Python program – Alpha-Beta Pruning

```
def alphabeta(values, alpha=float("-inf")):
    # Root is MAX; each move is a separate MIN node.
    best_move = None
    best_value = float("-inf")
    scored = 0
    skipped = []

    for move_name, leaves in values:
        node_value = float("inf")
        beta = float("inf")  # Reset beta for each MIN child of the root.

        for i, score in enumerate(leaves):
            scored += 1
            node_value = min(node_value, score)
            beta = min(beta, node_value)

            if beta <= alpha:
                skipped.extend(leaves[i + 1:])
                break

        if node_value > best_value:
            best_value = node_value
            best_move = move_name

        alpha = max(alpha, best_value)

    return best_move, best_value, scored, skipped


# Input values from Question 5
moves = [
    ("M1", [6, 9, 7]),
    ("M2", [4, 8, 10]),
    ("M3", [7, 5, 12])
]

# Call the function and display the results
move, value, scored, skipped = alphabeta(moves)

print("Chosen move:", move)
print("Value:", value)
print("Positions scored:", scored)
print("Skipped positions:", skipped)
```

Overall Result:

Question	Main result
Q1	Greedy Best-First Search; Gate → Canteen → Admin → Auditorium; 580 m; not optimal.
Q2	A* Search; Gate → Canteen → Hostel → Auditorium; 480 m; shortest route.
Q3	CSP / graph colouring; one valid timetable: W2,W5 Morning; W3,W4 Afternoon; W1 Evening.
Q4	Minimax; X chooses first move 1 and guarantees score 7.
Q5	Alpha-beta pruning scores 6 of 9 leaves; chooses M1 with value 6.